AI Assisted Coding Assignment- 18.3

RUDROJU RUPA SRI

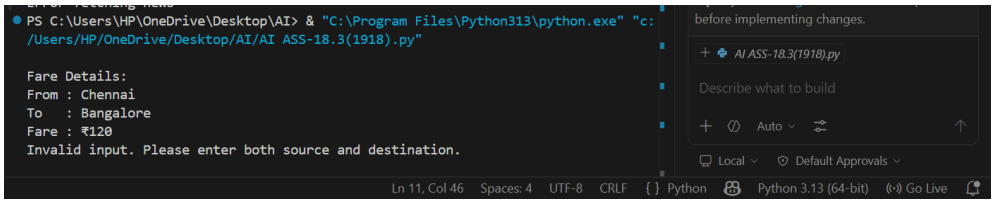
2303A51918

BATCH-30

Lab 18 – API Integration: Connecting to External Services with Error Handling
Lab Objectives:
• Learn how to integrate Python programs with external REST APIs.
• Understand API request/response handling using requests or similar libraries.
• Implement proper error handling for failed API calls (timeouts, invalid responses, rate limits).
• Practice extracting and displaying meaningful information from API responses.
Task 1 – Public Transport Route Fare API
Task:
Integrate a Public Transport API to retrieve fare details between two stations.
Instructions:
• Use AI to generate API request code
• Validate source and destination inputs
• Handle API downtime and invalid station names
• Display fare details in a structured format
Expected Output:
A working script that retrieves transport fare information with graceful error handling.

```
# Task-1: Public Transport Fare

def get_fare(source, destination):
    # Input validation
    if not source or not destination:
        print("Invalid input. Please enter both source and destination.")
        return

    # Convert to proper format (avoid case issues)
    source = source.strip().title()
    destination = destination.strip().title()

    # Dummy fare data (simulating API)
    fare_data = {
        ("Chennai", "Bangalore"): 120,
        ("Chennai", "Hyderabad"): 200,
```

```
        ("Delhi", "Mumbai"): 300
    }

    try:
        fare = fare_data.get((source, destination))

        if fare:
            print("\nFare Details:")
            print(f"From : {source}")
            print(f"To   : {destination}")
            print(f"Fare : ₹{fare}")
        else:
            print("Invalid station names or route not found.")

    except Exception as e:
        print("Something went wrong:", e)


# Test
get_fare("chennai", "bangalore")
get_fare("", "Delhi")
```



Task 2 – Currency Exchange Rates
Task:
Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.
Instructions:
• Use AI to generate API request code.
• Validate user input for amount and currency codes.
• Implement error handling for invalid responses and API downtime.
• Show conversion results clearly in tabular format.
Expected Output:
• A script that converts INR to multiple currencies with graceful error handling.

```
# Task-2: Currency Exchange

# Handle missing library
try:
    import requests
except ImportError:
```

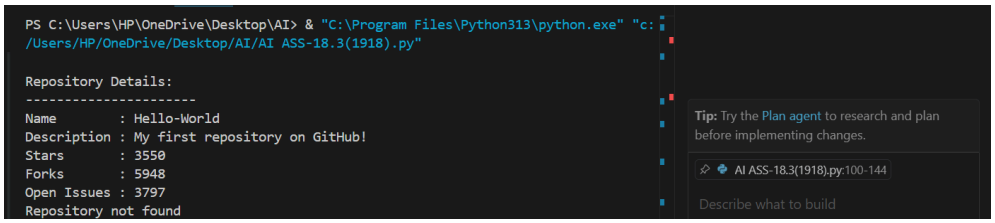
```
    print("Requests module not installed. Run: pip install requests")
    exit()

def convert_currency(amount):
    # Input validation
    if amount <= 0:
        print("Invalid amount. Please enter a positive value.")
        return

    url = "https://api.exchangerate-api.com/v4/latest/INR"

    try:
        response = requests.get(url, timeout=5)

        # Check API response
        if response.status_code != 200:
            print("API error. Status Code:", response.status_code)
            return

        data = response.json()

        # Extract rates safely
        rates = data.get("rates")
        if not rates:
            print("Invalid API response")
            return

        # Conversion
        usd = amount * rates.get("USD", 0)
        eur = amount * rates.get("EUR", 0)
        gbp = amount * rates.get("GBP", 0)

        # Display output in table format
        print("\nCurrency Conversion Table")
        print("----------------------------")
        print(f"{'Currency':<10} {'Amount'}")
        print("----------------------------")
        print(f"{'USD':<10} {usd:.2f}")
        print(f"{'EUR':<10} {eur:.2f}")
        print(f"{'GBP':<10} {gbp:.2f}")

    except requests.exceptions.Timeout:
        print("Request timed out. Try again.")
    except requests.exceptions.ConnectionError:
        print("Network error. Check your internet.")
    except Exception as e:
        print("Unexpected error:", e)


# Test
```

```
convert_currency(1000)
convert_currency(-50)
```

Task 3 – GitHub Repository Info Fetcher
Task:
Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).
Instructions:
• Prompt AI to write a Python function to send GET requests to GitHub API.
• Handle rate limit errors, 404 (repo not found), and invalid input.
• Display repository name, description, stars, forks, and open issues.
Expected Output:
• A Python program that retrieves and displays GitHub repository information with robust error handling.

```
#Task-3
import requests

def get_repo_info(repo):
    # Input validation
    if not repo or "/" not in repo:
        print("Invalid input. Use format: owner/repo")
        return

    try:
        url = f"https://api.github.com/repos/{repo}"
        response = requests.get(url, timeout=5)

        if response.status_code == 200:
            data = response.json()

            print("\nRepository Details:")
            print("----------------------")
            print(f"Name        : {data.get('name')}")
            print(f"Description : {data.get('description')}")
            print(f"Stars       : {data.get('stargazers_count')}")
            print(f"Forks       : {data.get('forks_count')}")
            print(f"Open Issues : {data.get('open_issues_count')}")
```

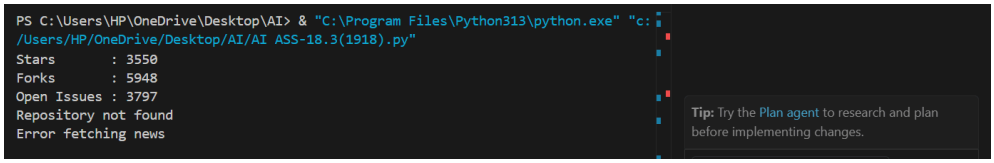
```
        elif response.status_code == 404:
            print("Repository not found")

        elif response.status_code == 403:
            print("Rate limit exceeded. Try later")

        else:
            print("Error:", response.status_code)

    except requests.exceptions.Timeout:
        print("Request timed out")
    except requests.exceptions.ConnectionError:
        print("Network error")
    except Exception as e:
        print("Unexpected error:", e)


# Test
get_repo_info("octocat/Hello-World")
get_repo_info("wrong/repo")
```



Task 4 – Real-Time Application: News Headlines Aggregator
Scenario:
Build a News Aggregator using a free News API.
Requirements:
• Fetch top 5 headlines for a given category (sports, technology, health).
• Handle errors like invalid category, missing API key, and request failures.
• Display headlines in a user-friendly numbered list format.
• Include a retry mechanism if the first request fails.
Expected Output:
• A working Python script that retrieves and displays news headlines with proper error handling

```
#Task-4
import requests
import time

API_KEY = "YOUR_API_KEY"
```

```
def get_news(category):
    url = f"https://newsapi.org/v2/top-headlines?category={category}&apiKey={API_KEY}"

    for attempt in range(2):  # retry mechanism
        try:
            res = requests.get(url, timeout=5)

            if res.status_code != 200:
                print("Error fetching news")
                return

            data = res.json()

            articles = data.get("articles", [])[:5]

            print("\nTop Headlines:")
            for i, article in enumerate(articles, 1):
                print(f"{i}. {article['title']}")

            return

        except requests.exceptions.RequestException:
            print("Retrying...")
            time.sleep(2)

    print("Failed after retry")

# Test
get_news("technology")
```



Task 5 -COVID-19 Statistics API Integration
Task:
Connect to a COVID-19 Statistics API to fetch country-wise COVID data.
Instructions:
• Use AI to generate Python code using the requests library
• Accept country name as user input
• Fetch key statistics such as:
o Total confirmed cases
o Total deaths
o Total recovered cases
o Active cases
• Handle API errors including:

o Invalid country name
o API rate-limit exceeded
o Network or server failures
• Implement retry or wait logic when rate limits are hit
Expected Output:
A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.

```
#Task-5
import requests
import time

def get_covid_data(country):
    url = f"https://disease.sh/v3/covid-19/countries/{country}"

    for attempt in range(2):
        try:
            res = requests.get(url, timeout=5)

            if res.status_code == 404:
                print("Invalid country")
                return
            elif res.status_code == 429:
                print("Rate limit hit, retrying...")
                time.sleep(2)
                continue

            data = res.json()

            print("\nCOVID Stats:")
            print("Country:", data["country"])
            print("Total Cases:", data["cases"])
            print("Deaths:", data["deaths"])
            print("Recovered:", data["recovered"])
            print("Active:", data["active"])

            return

        except requests.exceptions.Timeout:
            print("Timeout, retrying...")
        except requests.exceptions.RequestException:
            print("Network error")

    print("Failed to fetch data")

# Test
get_covid_data("India")
```


COVID Stats:
Country: India
Total Cases: 45035393
Deaths: 533570
Recovered: 0
Active: 44501823
PS C:\Users\HP\OneDrive\Desktop\AI>
AI ASS-18.3(1918).py:183-221
Describe what to build
Auto
Local
Default Approvals